

Private Tensor Memory Hierarchy

Arunan Elamaran Sourish Saswade

I. INTRODUCTION & MOTIVATION

In the current Vortex codebase, the `sgemm_tcu` benchmark performs matrix multiplication using the existing Tensor Core implementation and the WMMA ISA extension. In this design, matrix tiles are loaded from global memory into fragments backed by register arrays, after which `mma_sync` instructions operate on these register-stored tiles to compute partial products. The WMMA extension decomposes the instruction into multiple micro-steps based on tile dimensions, number of registers, and number of threads. Each step issues a distinct instruction that consumes a subset of the tile data stored in registers. This approach results in a large number of instructions per tile and places significant pressure on the register file due to repeated packing, unpacking, and reuse of micro-tile operands.

In this project, we implemented Tensor Memory (TMEM) as an architectural extension to Vortex to address these inefficiencies. TMEM is designed as a private, per-SM tensor scratchpad, modeled after Vortex’s existing local memory mechanisms, but specialized for tiled MMA workloads. Instead of fragmenting tiles across registers, TMEM stores entire tiles contiguously and enables MMA operations to reference operands and accumulators via memory addresses rather than register fragments. By moving tensor data out of the register file and into TMEM, TMEM substantially reduces register pressure and eliminates much of the overhead associated with register-based micro-tiling.

Building on TMEM, we added an ISA extension, UMMA, that enables larger, tile-level MMA operations with fewer instructions. UMMA reduces the number of issued instructions required to perform a full MMA by operating directly on TMEM tiles. This thus lowers instruction fetch and decode overhead. Together, TMEM and UMMA support direct memory-based access to tensor tiles, simplify data movement by avoiding complex register layouts, allow larger tile granularity, and also accommodate multiple input and output data types without requiring distinct ISA variants.

II. IMPLEMENTATION DETAILS

A. High-Level Architecture

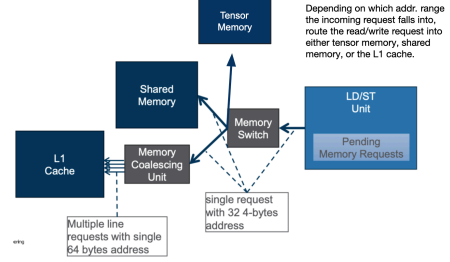


Figure 1: TMEM integration w/ existing LD/ST unit

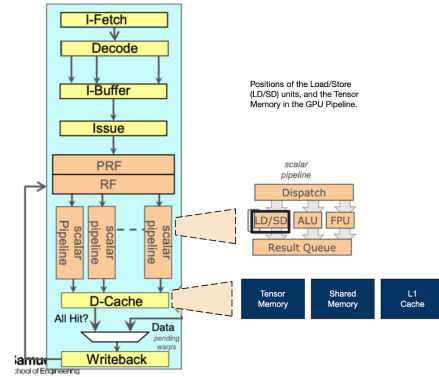


Figure 2: TMEM in the Vortex GPU Pipeline

In the two figures above, we have an architecture diagram that details where TMEM falls in the Vortex GPU pipeline (Figure 2), and how the TMEM unit is integrated with the existing LD/ST unit to serve TMEM read/write operations. In Figure 1, we can see that we altered the load/store unit in Vortex to route read/write requests into tensor memory depending on if the requested addresses fall into the tensor memory range that we define. In figure 2, within the pipeline, the load/store unit is located amongst the ALU and FPU and interacts with different memory units like the existing shared memory.

B. ISA Extensions (for TMEM & UMMA)

The main component of the extension is UMMA, a tensor-core instruction designed to operate directly on

TMEM- and SMEM-resident data. Operand A and B may be sourced from either shared memory (SMEM) or TMEM (but contain TMEM addresses in our code). The accumulator (C/D) is always read from and written back to TMEM. UMMA enables larger, tile-level operations with fewer instructions.

To enable TMEM operations, a read function defined in the TensorMem class models a load (ld) from TMEM, and a write function models a store (st) into TMEM.

Finally, instead of using an explicit TMEM allocation operation, we made the tensor memory base address visible at the software level, thus enabling direct reads of stores of data at a specified TMEM address. This reflects TMEM's role as a private, explicitly managed scratchpad rather than a general-purpose heap.

C. Memory Interconnect for TMEM

To make TMEM accessible to executing kernels in the same way as local memory could be used, we extended the SimX memory interconnect to treat TMEM as a destination for LSU memory requests. Rather than introducing a separate path, TMEM was integrated into the existing load/store pipeline so that requests targeting tensor memory are routed, arbitrated, and serviced using the same structural mechanisms already used for global memory and local (shared) memory.

First we introduced a new address classification, `AddrType::Tensor`, into the SimX address decoder. Any address falling within the configured TMEM address range is now identified as targeting tensor memory. This classification step easily enables downstream components to route memory operations to TMEM. From the perspective of the other instructions, TMEM accesses are still standard load/store operations.

Afterwards, we extended the existing LocalMemSwitch to support TMEM as an additional routing destination. Previously, this component acted as a two-way switch, forwarding each lane of an LSU request either to local memory (LMEM) or to the global data cache path. With TMEM support, the switch becomes three-way. For each active lane in a vector LSU request, the switch now determines whether the target address corresponds to shared memory, tensor memory, or global memory, and forwards the request accordingly. This required

adding new request and response ports dedicated to TMEM, while preserving the existing merging logic that recombines responses back into a single LSU response stream.

At the core level, TMEM is instantiated and wired similarly to how it is done for the existing local memory subsystem. A TensorMem object is created to model the physical storage and access behavior of TMEM. To connect this memory to the LSU pipeline, a dedicated arbitration and adaptation path was introduced. Specifically, each LSU block forwards its tensor memory requests to a shared TMEM arbiter.

We modeled TMEM as a banked, on-chip memory structure. We use a simple addressing scheme where we allocate a range of memory addresses for TMEM. It contains a RAM array sized according to the configured TMEM capacity, with addresses translated from global tensor addresses into local offsets. A crossbar distributes incoming requests across banks based on address bits, allowing multiple accesses to proceed in parallel when they target different banks.

C. UMMA Kernel (kernel.cpp)

The kernel in `tests/regression/final/kernel.cpp` implements a tiled matrix multiplication (GEMM) that uses TMEM as the primary storage for both operands and accumulators. Each thread block is responsible for computing a single output tile of matrix C, with the tile's row and column determined by the block's two-dimensional grid indices. This mapping allows the overall GEMM to be decomposed into independent tile-level computations, each handled by one block.

At the start of execution, the kernel computes the sizes of the A, B, and accumulator tiles and assigns each block a private, contiguous region of TMEM. This region is statically partitioned so that one portion stores the A tile, another stores the B tile, and the final portion holds the accumulator tile. Rather than storing tensor data in registers, the kernel uses fragments that contain TMEM addresses pointing to these regions. The accumulator tile in TMEM is explicitly initialized to zero before computation begins.

The core of the kernel consists of a loop over the K dimension, where the computation proceeds in tile-sized increments. In each iteration, a tile of matrix

A and a corresponding tile of matrix B are loaded from global memory directly into TMEM using tiled load operations. The addressing and layout of the B tile depend on the input data type: for sub-byte types, B is treated as column-major, while for byte-sized and larger types it is treated as row-major. Once both tiles are placed in TMEM, the kernel invokes an MMA operation that reads operands A and B from TMEM and accumulates the result into the TMEM accumulator tile. This MMA step operates entirely on TMEM-stored data, allowing accumulation without fragmenting the tile across multiple micro-operations.

After all K tiles have been processed and the accumulation is complete, the final accumulator tile is written back from TMEM to global memory as the corresponding output tile of matrix C.

D. Decode (*decode.cpp*)

The source registers are set as a0, a1, and a2, and contain the addresses of A, B, and C in tensor memory. The destination register is set as the address of C as well as it is the accumulator. UMMA is able to push a single instruction into the instruction buffer because it takes tile addresses in TMEM as input and thus executes the MMA at the tile level. On the other hand, since WMMA must pack actual tensor data, not addresses, into 32 floating-point registers, it must split up the MMA into multiple micro-steps and thus pushes many instructions into the buffer. This highlights one of the primary benefits of TMEM: reduced instruction count and thus reduced instruction overhead.

E. Execute (*execute.cpp*)

The execute stage happens in the tensor core unit. First, we gather the base addresses of the matrices that were passed into the source registers of the instruction. Then, we get the tile dimensions, and depending on the input and output types of the matrix multiplication, we select the appropriate executor function. We wrote a templated execute function that reads in the source A, B, and C tiles from tensor memory, completes the MMA using the FMA function, and then writes the C tile back to tensor memory. Lastly, the address of the destination tensor is returned to rd for all threads, so they can use it to store the output of the tiled MMA from tensor memory to global memory.

The execute stage highlights the other primary benefit of tensor memory: reduced register file pressure, as we are reading from and writing to the tensor memory unit rather than the 32 registers required per instruction for WMMA. Reduced register file pressure has a few benefits, one being that the registers are freed up for use by other threads or applications that may need them.

III. EVALUATION & RESULTS

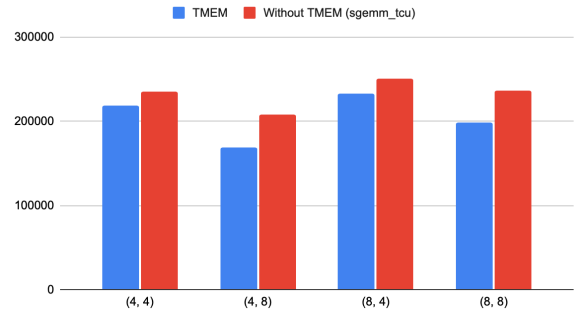


Figure 3: Instruction Count vs. (Warps, Threads)

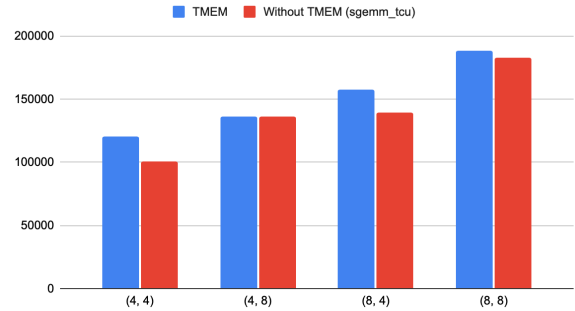


Figure 4: Cycle Count vs. (Warps, Threads)

(W, T)	Instructions (%)	Cycles (%)	IPC (%)
(4, 4)	-7.39	+19.89	-22.76
(4, 8)	-18.75	-0.34	-18.47
(8, 4)	-6.96	+13.34	-17.91
(8, 8)	-16.43	+3.07	-18.92

Figure 5: % Difference in Performance from baseline (non-TMEM) to TMEM

In Figure 3 and Figure 4 we have visualized a comparison between our tensor memory implementation and a baseline `sgemm_tcu` non-tensor-memory implementation. We plotted both instruction count and cycle count. As expected, tensor memory offers a decreased instruction count across the board. This makes sense, as we are not producing multiple micro-ops per instruction in UMMA the way WMMA does. Additionally, on average the cycle count for TMEM is a bit greater than the cycle count for non-TMEM. This was an interesting result to us but based on our current implementation, it is a result of the additional read/write latencies that each MMA operation goes through in the execute stage. This would provide the strong benefit of reduced register file pressure if implemented in reality, but the cycle count tells us how constant access to the register file is faster than constant access to the tensor memory.

Figure 5 tells us about how the averages for instruction count (IC), cycle count (CC), and IPC changes when going from the `sgemm_tcu` baseline to our TMEM implementation. TMEM exhibits a 12.38% decrease in IC, an 8.99% increase in CC, and a 19.51% decrease in IPC. We made two interesting observations.

1) Instruction count is lower with more threads (8): UMMA generates one instruction per tiled MMA operation, and WMMA generates $m*n*k$ microsteps. The tile cap is defined as the number of registers times the number of threads. Thus, more threads means larger tiles meaning more $m/n/k$ steps and micro-ops in WMMA, while UMMA remains at one instruction. Thus, UMMA scales better with more threads.

2) Cycle count much higher with fewer threads (4): With fewer threads, we have smaller tiled and thus more iterations of the k-loop that iterates through tiles. Each iteration needs to do loads and stores to and from tensor memory. Thus, more iterations means more loads and stores, a greater memory overhead and thus greater cycle count.

IV. ARTIFACTS EVALUATION

A. Github (<https://github.com/ArunanElamaram/vortex>)

The **fully working implementation for simx** is in the ‘simx’ branch. The completed but not yet fully working

rtlsim implementation (**compiles successfully but encounters a scoreboard bug while running the regression test**) is in the ‘rtlsim’ branch. Both Arunan and Sourish worked together for both simulators. Branches “simxAlternate” and “rtlAttemptOld” were used to enable parallel development and also experiment with alternate solutions for simx and rtlsim.

B. Instructions to Reproduce Results

Ensure all basic Vortex setup commands detailed in the Vortex README have been completed first, including ensuring the vecadd demo runs.

(<https://github.com/vortexgpgpu/vortex>)

Command 0 (Must be in build directory before running make/run commands): `cd /vortex/build`

Command 1a (make test ‘final’, includes TMEM):

```
make -C tests/regression/final clean &&
CONFIGS="-DNUM_THREADS=4 -DITYPE=fp16
-DOTYPE=fp32" make -C tests/regression/final
```

Command 1b (run test ‘final’):

```
CONFIGS="-DNUM_THREADS=4
-DEXT_TCU_ENABLE" ./ci/blackbox.sh --cores=4
--warps=4 --threads=4 --driver=simx --app=final
--args="-n 128"
```

Command 2a (make test ‘sgemm_tcu’, no TMEM):

```
make -C tests/regression/sgemm_tcu clean &&
CONFIGS="-DNUM_THREADS=4 -DITYPE=fp16
-DOTYPE=fp32" make -C tests/regression/sgemm_tcu
```

Command 2b (run test ‘sgemm_tcu’):

```
CONFIGS="-DNUM_THREADS=4
-DEXT_TCU_ENABLE" ./ci/blackbox.sh --cores=4
--warps=4 --threads=4 --driver=simx --app=sgemm_tcu
--args="-n 128"
```

To test with different input types and output types, simply change the DITYPE and DOTYPE options. To change threads (we used 4 and 8), simply change the DNUM_THREADS option. The “n” option specifies the n dimension size of the input matrices. We tested with n=128, but this can be adjusted as well.